

Camada ORM e API pública

Mapeamento SQLAlchemy, sessões, relacionamentos e contrato HTTP da aplicação atual

Uma única camada pública

A aplicação foi consolidada em uma API com SQLAlchemy. Não existe uma segunda família de rotas para comparar implementações. O OpenAPI atual contém **37 caminhos e 64 operações**, contando a raiz, e o frontend chama esses mesmos caminhos. Essa decisão removeu duplicação de schemas, tratamento de erro e regras de atualização.

Arquivo	Responsabilidade
modelos.py	Treze entidades e seus relacionamentos.
database.py	Engine, fábricas de sessão e DATABASE_URL.
routers/*.py	CRUD e operações específicas por recurso.
routers/comum.py	Commit, rollback e conversão de erros do banco.
consultas.py	Consultas do item 5 e medição lazy/eager.
analiticas_db.py	Quatro consultas analíticas recuperadas da Etapa 1.

O engine não chama `create_all()`. O esquema continua sob responsabilidade dos arquivos SQL. Assim, constraints, triggers, views e rotinas armazenadas têm uma única fonte de definição. A sessão ORM também pode executar CALL e SELECT textual quando o alvo é um objeto próprio do banco.

Treze entidades mapeadas

Pessoa, Paciente, Alergia, Profissional, Preceptor, Residente, Unidade, Atendimento, Procedimento, ProcedimentoRealizado, Escala, Internacao e AuditoriaAtendimento formam o mapeamento. **Papel_Profissional** não aparece como entidade de negócio: os triggers a mantêm como mecanismo interno de exclusividade. Soma e quantidade da média também ficam sob controle do banco.

Especializações como relacionamentos

Pessoa pode ter vínculo de paciente, profissional ou ambos. Por isso Paciente e Profissional são relacionamentos um-para-um, com `uselist=False`, e não subclasses polimórficas. Profissional se relaciona da mesma forma com Preceptor e Residente, mas a chave compartilhada em `Papel_Profissional` impede que os dois papéis estejam ativos ao mesmo tempo.

CPF e CRM foram declarados sem aspas no SQL e o PostgreSQL os guarda em minúsculas. O atributo Python preserva o nome esperado pela API, enquanto `mapped_column` informa o nome físico. Essa tradução fica no modelo e não precisa ser repetida nas consultas.

Chaves e ciclos de vida

`ProcedimentoRealizado` usa a chave composta (`id_atendimento, id_procedimento`); `db.get` recebe essa tupla na mesma ordem. Coleções dependentes usam `cascade` e `delete-orphan` quando a remoção faz parte do ciclo de vida do pai. Ao excluir um paciente, a linha de `Pessoa` só é apagada se não houver vínculo profissional, evitando remover outro papel legítimo.

Sessão, commit e respostas de erro

`get_orm_db` abre uma sessão por requisição e a fecha no `finally`. Cada router decide quando confirmar. **`confirmar()`** faz rollback em qualquer falha e converte SQLSTATE conhecidos: unicidade vira 409; chave estrangeira, nulidade, CHECK e parâmetro inválido viram 400. Recursos ausentes recebem 404 antes da gravação.

Conflito otimista. `StaleDataError` não é escondido como erro genérico. O auxiliar desfaz a transação e deixa o router de escala converter o conflito em HTTP 409.

Carregamento sob demanda e adiantado

Relacionamentos começam sob demanda. Isso é simples quando o dado relacionado raramente será lido, mas uma listagem pode produzir N+1 consultas. As rotas escolhem a estratégia de acordo com a forma da resposta.

Estratégia	Uso no projeto
joinedload	Traz relações do lado um no mesmo SELECT, como atendimento e paciente.
selectinload	Carrega coleções com uma segunda consulta e IN, sem multiplicar a consulta principal.
contains_eager	Reaproveita um JOIN que já foi necessário para filtrar ou ordenar.

GET /consultas/lazy-vs-eager esvazia o mapa de identidade, percorre os atendimentos nas duas estratégias e conta instruções no engine. O teste exige resultados iguais e menos consultas no modo adiantado; não fixa um número para o modo lazy, pois ele depende dos relacionamentos distintos presentes na massa de dados.

Versão lida pelo cliente

Escala declara `versao` como `version_id_col`. O cliente recebe esse valor na leitura e precisa devolvê-lo no **PUT /escalas/{id_escala}**. O router compara a versão enviada com a atual antes de alterar. Se outra transação gravar depois dessa comparação, o próprio UPDATE gerado pela ORM ainda inclui a versão no WHERE e levanta `StaleDataError` quando afeta zero linhas.

```
UPDATE escala
  SET id_unidade = :nova_unidade, versao = :versao_nova
 WHERE id_escala = :id AND versao = :versao_lida;
```

As duas verificações cobrem conflitos conhecidos antes do flush e a janela de corrida durante a gravação. Em ambos os casos, a resposta é 409 e orienta o cliente a recarregar a escala.

Rotas fixas antes de identificadores

Starlette avalia os caminhos na ordem de registro. Por isso rotas como **/pacientes/internados**, **/atendimentos/estatisticas-mensais** e **/escalas/reajustar** aparecem antes de **/id**. O `TestClient` cobre oito caminhos literais que poderiam ser confundidos com identificadores.

Criação de atendimento

A consulta e a manutenção comum usam **/atendimentos**. Para criar um caso válido com sua lista de procedimentos, o fluxo público é **POST /atendimentos/completo**. Um atendimento isolado não passa pelo COMMIT, pois o constraint trigger exige ao menos um procedimento. A interface direciona o cadastro para a operação completa.